

数十万人同時に英語スピーキングテストを 実施するための CBT のアーキテクチャ

Paul Grudnitski¹, 村井拓海², 佐々木公博², 村田真³

¹Amp-up.io 社, ²株式会社ネットラーニング, ³東日本国際大学経済経営学部

1. はじめに

近年, 全国学力・学習状況調査において英語スピーキングテストの導入が進められており, 約 90 万人の中学三年生がその対象となる. また, 共通テストに導入する場合には, リーディング・リスニングと同様に約 45 万人が受験する. しかし, これまで大学入試での CBT スピーキング導入例はあるが規模は数百人にとどまる. 海外にも数十万人同時の CBT スピーキングテストの事例は見当たらない.

2026 年度の全国学力・学習状況調査の英語スピーキングは 1 か月にわたって分散実施されるが, 共通テストでは問題漏洩防止のため全国同時実施が前提となる. したがって, 数十万人規模の同時接続に耐える CBT システムが不可欠である.

数百人規模の CBT システムを拡張しても, 数十万人規模に対応することは困難である. 小規模システムの単純な拡張では大規模に対応できないことは, Web アプリケーションスケラビリティ研究 (Williams & Smith, 2004; Olston et al., 2005; Iyer et al., 2005) でも指摘されている. 数百人規模の CBT システムを 1000 個並べて使えば数十万人規模に対応できると単純に考える人もいるかも知れないが, 実際にはそれらを連動させるための管理コスト (CBT 問題の分配, 解答データの集約も含む) が新たに発生するので話はそう簡単ではない. また, ネットワークを共有すればそこがボトルネックになって動作しない可能性もある.

本研究では, 数十万人同時にスピーキングテストを実施するために, 6 つの特徴を備えた CBT アーキテクチャを提案する. (1) 問題配信と解答収集の分離, (2) オートスケールとロードバランシングの活用, (3) 前処理による問題配信の効率化, (4) クライアント側での進行管理によるステートレス化, (5) 解答データベース処理の単純化 (INSERT のみ), (6) Pod 単位での受験者割当によるデータベース負荷分散.

本アーキテクチャに基づく CBT を実装し, 性能評価を行った. 1 時間未満で 35 問のテストを完了する同時 20 万人の JMeter シミュレーションで安定動作を確認した. さらに一桁上, すなわち 1 時間未満で 35 問を完了する同時 200 万人規模まで確実にスケールアップ可能と考えられる.

2. 提案する CBT アーキテクチャの概要

2.1 問題配信サーバ群と解答収集サーバ群の分離

従来の国内の CBT では, 1 つのサーバ群が問題配信と解答収集の両方を担当することが多い. しかし, 問題配信に求められる機能・性能と, 解答収集に求められる機能・性能はまったく異なるので, 両方を同一のサーバ群に委ねると最適化を十分に行うことができない.

本研究では, 問題配信専用サーバ群と解答収集専用サーバ群を分離する. この分離により, 問題配信サーバ群は問題を受験者に高速に配信することに特化した最適化が, 解答収集サーバ群は受験者から解答を収集して解答データベースに書き込むことに特化した最適化が可能になる. 問題配信サーバ群には, 一般的な Web 配信の最適化手法 (キャッシュなど) がそのまま有効である.

2.2 オートスケールとロードバランシングの活用

本研究では、受験者数の変動に応じてサーバ群の規模が自動的に増減するようオートスケールリングを設定する。さらにロードバランサにより各サーバへのリクエストを均等に分配することで負荷を分散する。

問題配信サーバ群と解答収集サーバ群はそれぞれ別にオートスケールする。問題配信サーバ群は試験開始直後の最大負荷に耐えるようオートスケールするが問題の配信が終われば試験終了前であっても徐々に撤退していく。一方、解答収集サーバ群は試験終了直前の最大負荷に耐えるようオートスケールし、試験終了後に一気に撤退する。

2.3 前処理による問題配信の効率化

CBT 問題は、外部リソースへの参照を含み、そうした参照は相対 URL によることもある。本研究の問題配信サーバは、配信前に問題コンテンツの整合性を検証したうえで、展開できる外部リソース参照は展開し、相対 URL は絶対 URL に変換して静的ファイルを作成しておく。生成された静的ファイルはキャッシュの利用が容易であり、大規模同時配信時の安定性と効率性を高める。テスト実行時には、問題配信サーバは静的ファイルの配信だけを行う。

2.4 クライアント側での進行管理によるステートレス化

CBT 問題をクライアント端末上で解釈し実行するプログラムを CBT プレーヤと呼ぶ。本研究の CBT プレーヤは、テストの進行状況（どのテストの何問目をいま解いているか、すでに解答した問題はどれか等）を管理する。CBT プレーヤから解答収集サーバ群にリクエストを送る際には、進行情報を用いて自己完結的なリクエストを組み立てる。リクエストが自己完結であるとは、それを処理するために必要なすべての情報を含んでいることを意味する。自己完結的なリクエストは、サーバ側に保存された情報を用いずに解釈できる。

解答収集サーバは、各リクエストを処理し、解答とテストの進行状況とを解答データベースに保存する。しかし、各リクエストの処理後は、解答データベースにあるもの以外の情報を破棄し、次のリクエストにまで持ち越さない。すなわち解答収集サーバはステートレスである(Fielding 2000)。解答データベースにはテストの進行状況が保存されるが、それらがテストの実行中にリクエスト解釈のために使われることはない（異常終了時の回復処理を除く）。

ステートレス化により、オートスケールとロードバランシングを最大限に活用できる。さらに、クラウド基盤の冗長構成や自動フェイルオーバーが容易になり、大規模同時接続時にも高い安定性と可用性を確保できる。

2.5 解答データベース処理の単純化（INSERT のみ）

本研究では、解答データを解答データベースに格納する際は、単純な INSERT 処理のみで済むようスキーマを設計する。複雑な問い合わせや更新処理を伴う設計では、すぐにデータベースがボトルネックとなり、全体の性能低下や誤動作を招く。なお、この特徴は本研究に固有なものではなく既存 CBT システムでも採用されている。

2.6 Pod 単位での負荷分散

本研究では、解答収集サーバ群を複数の Pod に分割する。1 つの Pod には解答データベースサーバがただ1つまた必ず1つ存在する。ロードバランサーについても同様である。一方、解答収集 HTTP サーバは1つの Pod に複数存在する。Pod 内のすべての解答収集 HTTP サーバは同一の解答データベースに解答を格納する。

1 つの Pod にすべての受験者を集中させる構成では二、三十万人程度で解答データベースが処理限界に達するため、それ以上の人数をさばくためには複数の Pod を用いることが不可欠である。

本研究では、試験開始時に各受験者を Pod に割り当てる。Pod 割り当てのためのサーバを Switch Board と呼ぶ。受験者が Switch Board にアクセスするのは、原則として試験開始時だけに限られる。受験者は試験の最初から最後まで同じ Pod にアクセスするため、Pod 間での相互作用はない。必要な Pod の数は少ないので、解答データを集約するコストはごく小さい。

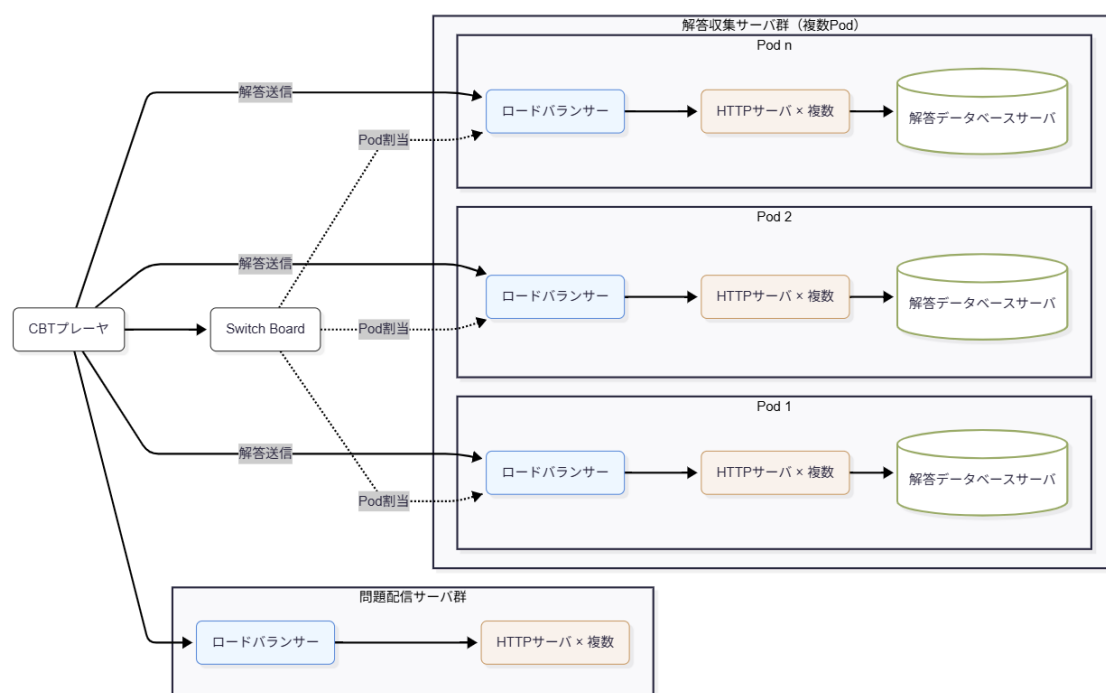


図 1. 本論文の CBT アーキテクチャ

3. 実装

本アーキテクチャを AWS (Amazon Web Services) 上で実装した。

問題配信には Amp-up 社の Contents Bank を採用した。Contents Bank を構成する HTTP サーバ群はオートスケーリングにより増減し、Application Load Balancer (ALB) によって負荷分散する。配信効率を高めるための CBT 問題の前処理も備えている。

CBT プレーヤとして Amp-up 社の QTI 3 Player を採用した。QTI 3 Player はテストの進行管理をクライアント側で行う。

解答収集サーバ群は株式会社ネットラーニングが開発した。本サーバ群は ECS Fargate 上に構築し、Service Auto Scaling 機能によってコンテナ数を動的に増減させた。受験者からのアクセスは ALB によって複数のコンテナに分散させた。各コンテナでは Nginx を HTTP サーバとして稼働させ、受信した解答データを C# プログラムで処理して解答データベースに格納した。解答データベースには Amazon RDS for MySQL を採用した。

受験者を特定の Pod に割り当てる Switch Board は、ECS Fargate で動作する DynamoDB を用いて C# で実装した。

4. 性能評価

全国規模のスピーキングテストを想定し、性能評価を実施した。評価にはオープンソースの性能試験ツール JMeter を用い、多数の仮想受験者をシミュレートしてサーバ負荷および応答時間を測定した。

12 分の間に 20 万人が受験し、各受験者は 30 秒（約 1 MB）の音声データを 4 回提出するというシナリオで実験した。総計すると、約 800GB の音声データを収集し、データベースに格納したことになる。解答収集 HTTP サーバ群は、4 vCPU/8 GB コンテナ×24 台、解答データベースは db.r6i.4xlarge ×2 の構成で運用した。

測定の結果、平均応答時間は 0.083 秒であり、音声データの取りこぼしは一切発生しなかった。以上より、本研究で提案した CBT アーキテクチャは、全国規模のスピーキングテストに必要な性能を達成できることが確認された。

5. 今後の課題

本研究の性能評価ではネットワーク環境を考慮していないが、実際の大規模スピーキングテストでは終了直前に多数の受験者が同時に音声データを送信するため、回線が混雑する可能性が高い。このような混雑時でも、CBT 停止やデータ損失などの致命的な障害を回避する仕組み（送信の分散や再送制御）が必要であるが、それらについての具体的な検討は今後の課題である。

引用文献

- Cronbach, L. J. (1951). Coefficient alpha and the internal structure of tests. *Psychometrika*, 16(3), 297-334. <https://www.doi.org/10.1007/BF02310555>
- Williams, L., & Smith, E. (2004). Web Application Scalability: A Model-Based Approach. Proceedings of the 5th International Conference on Web Engineering (ICWE 2004), pp. 219-232. Springer. https://doi.org/10.1007/978-3-540-27834-4_18
- Olston, C., Jiang, J., & Widom, J. (2005). Adaptive Filters for Continuous Queries over Distributed Data Streams. In: Proceedings of the 2005 Conference on Innovative Data Systems Research (CIDR 2005), pp. 19-31. <https://www.cidrdb.org/cidr2005/papers/P05.pdf>
- Iyer, S., Gupta, P., & Johri, P. (2005). Performance, Scalability and Reliability Issues in Web Applications. International Journal of Computer Engineering and Technology (IJ CET), 5(3), 227-234. https://libres.uncg.edu/ir/uncg/f/L_Iyer_Performance_2005.pdf
- 1EdTech Consortium. (2024). *QTI 3.0 Best Practices and Implementation Guide (BPIG)*, Version 2.0, released October 1, 2024. <https://www.imsglobal.org/spec/qti/v3p0/impl>
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
- Apache JMeter. (2025). *User's Manual*. Apache Software Foundation. <https://jmeter.apache.org>
- Amp-up.io QTI 3 Item Player Component for Vue 3, (2025). *GitHub*. <https://github.com/amp-up-io/qti3-item-player-vue3>